

FTRIA-002 — RI-Preserving Transformation

Part I — Foundations of Runtime-Invariant Degrees of Freedom

Sizhe Tan

with chatGPT (OpenAI)

2026-07-15

Repository: <https://github.com/sizhet/Function-Tunnel-and-Runtime-Invariant-Algebra-FTRIA>

Abstract

A Runtime Invariant (RI) is valuable not because it remains completely unchanged, but because it remains identifiable while numerous aspects of its implementation evolve.

This observation leads to one of the central concepts of the Function Tunnel Runtime Invariant Architecture (FTRIA):

Not every transformation changes the Runtime Invariant.

Many transformations preserve the invariant while modifying its implementation, organization, optimization, or representation.

This paper introduces the concept of **RI-Preserving Transformations**, which defines the family of allowable transformations under which a Runtime Invariant maintains its identity.

Rather than viewing software evolution as arbitrary code modification, FTRIA interprets evolution as movement inside an invariant-preserving transformation space.

1. Runtime Invariants Are Not Static Objects

Traditional software often treats correctness as preserving identical code.

However, real computational systems evolve continuously.

Examples include:

- code refactoring,
- compiler optimization,
- hardware migration,
- distributed deployment,
- model compression,
- prompt optimization,
- runtime scheduling.

These modifications frequently change the implementation without changing

the computational objective.

The Runtime Invariant therefore survives implementation changes.

2. What Is an RI-Preserving Transformation?

An **RI-Preserving Transformation** is any transformation that changes the implementation while preserving the Runtime Invariant.

Symbolically,

Implementation A

|



RI-Preserving Transformation

|



Implementation B

$$RI(A) = RI(B)$$

The observable realization changes.

The invariant does not.

3. Typical RI-Preserving Transformations

Common examples include:

- function decomposition,
- function composition,
- module reorganization,
- calling graph optimization,
- code refactoring,
- compiler optimization,
- cache optimization,
- hardware-specific optimization,
- distributed execution,
- pipeline scheduling,
- instruction substitution,
- semantic-preserving translation,
- implementation replacement.

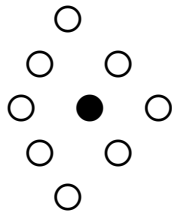
Although these transformations appear different from a software engineering perspective, they share one structural property:

The Runtime Invariant remains unchanged.

4. RI Preservation Defines Degrees of Freedom

A Runtime Invariant does not correspond to a single implementation.
Instead,
it corresponds to an entire family of admissible implementations.

Runtime Invariant



Each point represents a different implementation.
The center represents the Runtime Invariant.
Movement inside this region preserves the invariant.
Leaving the region changes the Runtime Invariant itself.
Therefore,
the allowable region constitutes the **Degrees of Freedom (DoF)** of the Runtime Invariant.

5. Preservation Is Structural Rather Than Syntactic

Two implementations may share almost no common source code while still preserving the same Runtime Invariant.

Conversely,

two implementations may look nearly identical syntactically while representing different Runtime Invariants.

Therefore,

RI preservation should be evaluated structurally rather than textually.

The identity of an RI depends on preserved structural capability, not implementation similarity.

6. Relation to Existing FTRIA Concepts

This principle naturally unifies several ideas developed throughout the Structural Intelligence framework.

Examples include:

- CCC-preserving generation,
- Differential transformations,
- Runtime migration,
- Calling Graph evolution,
- Function Tunnel optimization,
- Runtime packaging,

- Runtime reuse,
- Runtime projection.

Each of these can be interpreted as RI-preserving transformations operating within different Runtime-Invariant Degrees of Freedom.

This observation suggests that many seemingly independent algorithms are manifestations of a common invariant-preserving principle.

7. Toward Runtime Algebra

Once RI-preserving transformations are formally defined, they become composable.

Instead of manipulating code,

future computational systems may manipulate transformations themselves.

This leads naturally toward

- Runtime Algebra,
- Runtime Optimization,
- Runtime Compilation,
- Runtime Verification,
- Runtime Migration,
- Runtime Synthesis.

In this view,

software engineering evolves from programming implementations to navigating invariant-preserving transformation spaces.

Key Takeaways

- Runtime Invariants are preserved across many implementation changes.
- RI-preserving transformations modify implementations without changing invariant identity.
- A Runtime Invariant corresponds to a family of admissible implementations rather than a single realization.
- Degrees of Freedom arise from the set of all RI-preserving transformations.
- Many existing optimization, migration, translation, and refactoring techniques can be interpreted as instances of RI-preserving transformations.
- RI-preserving transformations provide the operational foundation for future Runtime Algebra.

Related FTRIA Documents

- **FTRIA-001 — Runtime Invariant Degrees of Freedom**
- **FTRIA-003 — Admissible Runtime Variation**

- **FTRIA-004 — Constraint Surfaces and Invariant Boundaries**
- **FTRIA-005 — Local and Global Degrees of Freedom**